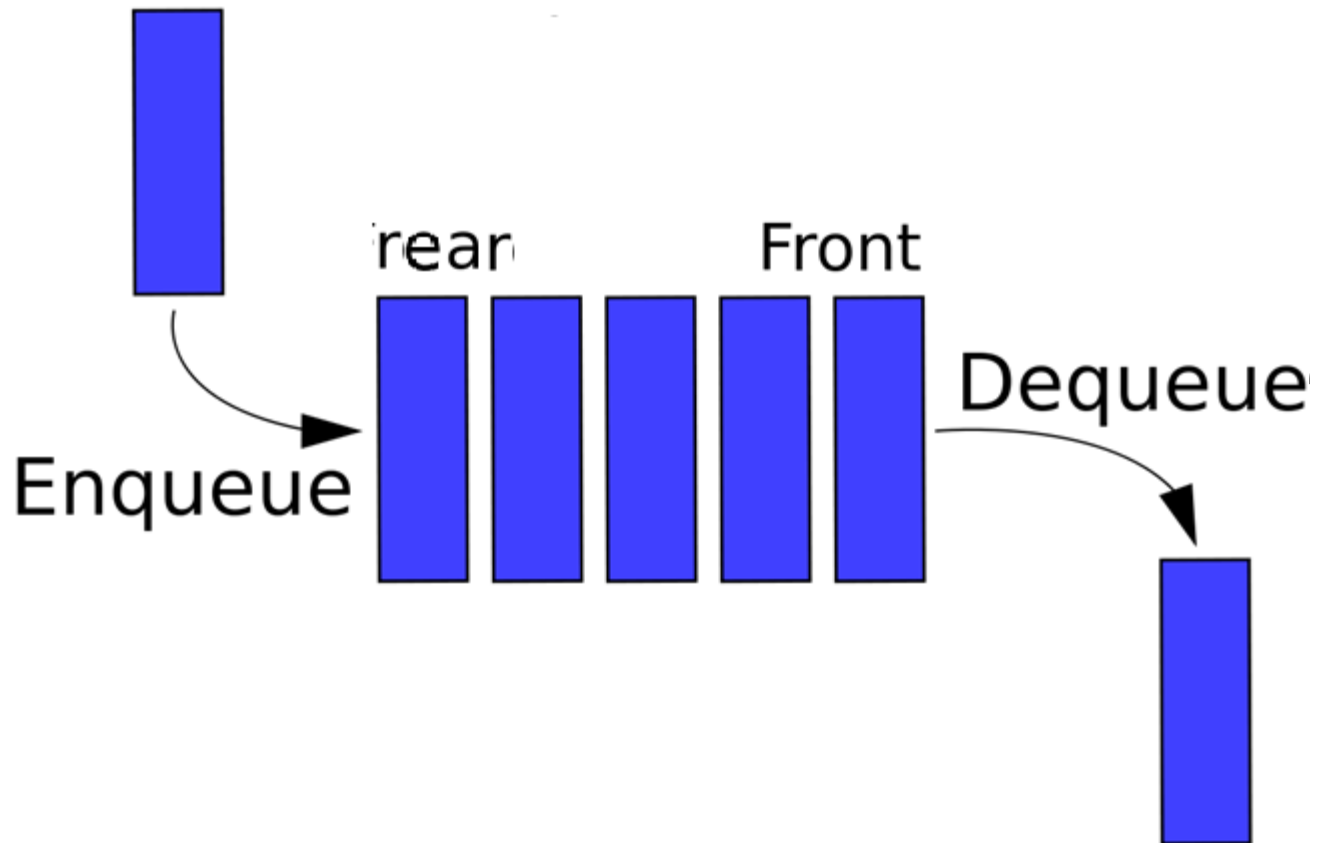


Fundamental data structures

Queue

Queue



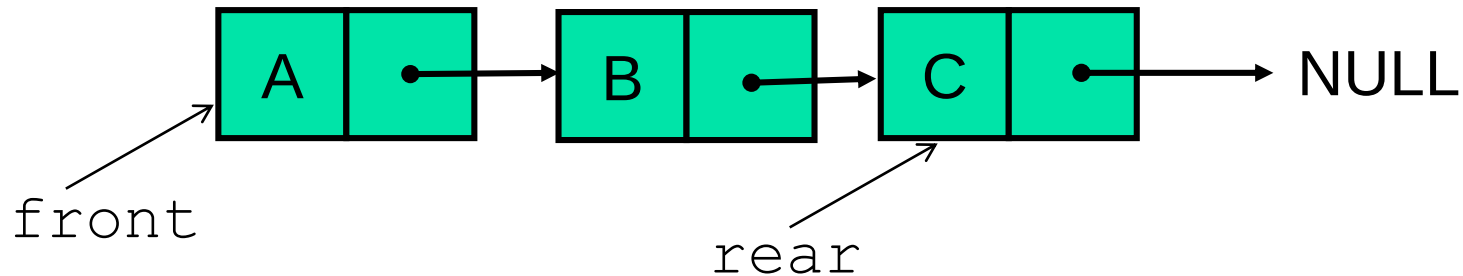


What is a Queue?

- A queue is an abstract data structure that only allows items to be inserted at the back(end) and remove from the front
- Queues are **FIFO** (First In First Out) data structures – “fair” data structures

Queue – front and back

- A queue has a **front** and a **rear**. Insertions occur at the rear and deletions at the front.





Applications of queue

- Printer spooling
- Task scheduling in OS
 - Process in ready queue are executed by CPU in FIFO order
- Breadth First Search (BFS)
- Order processing
 - e-commerce platform
 - ticket counter



Queue operations

- Main operations
 - Enqueue and Dequeue
- Other queue operations
 - initialize() - Initializes the queue
 - isEmpty() - Returns TRUE if the queue is empty.
 - isFull() - Returns TRUE if the queue is full.
- Implementation
 - Either arrays or linked list



Array Implementation

- A queue can be implemented with an array, as shown here.

[0]	[1]	[2]	[3]	[4]	[5]	...
4	8	6				

An array of
integers to
implement a
queue of integers

We don't care what's in
this part of the array.



Array Implementation (2)

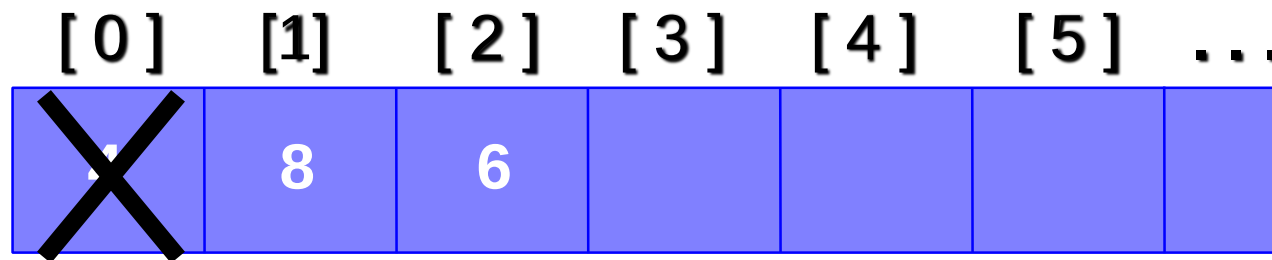
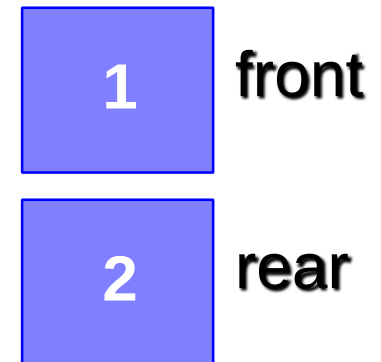
- Keep track of the front, rear and number of items in the queue.

0	front
2	rear

[0]	[1]	[2]	[3]	[4]	[5]	...
4	8	6				

Dequeue Operation

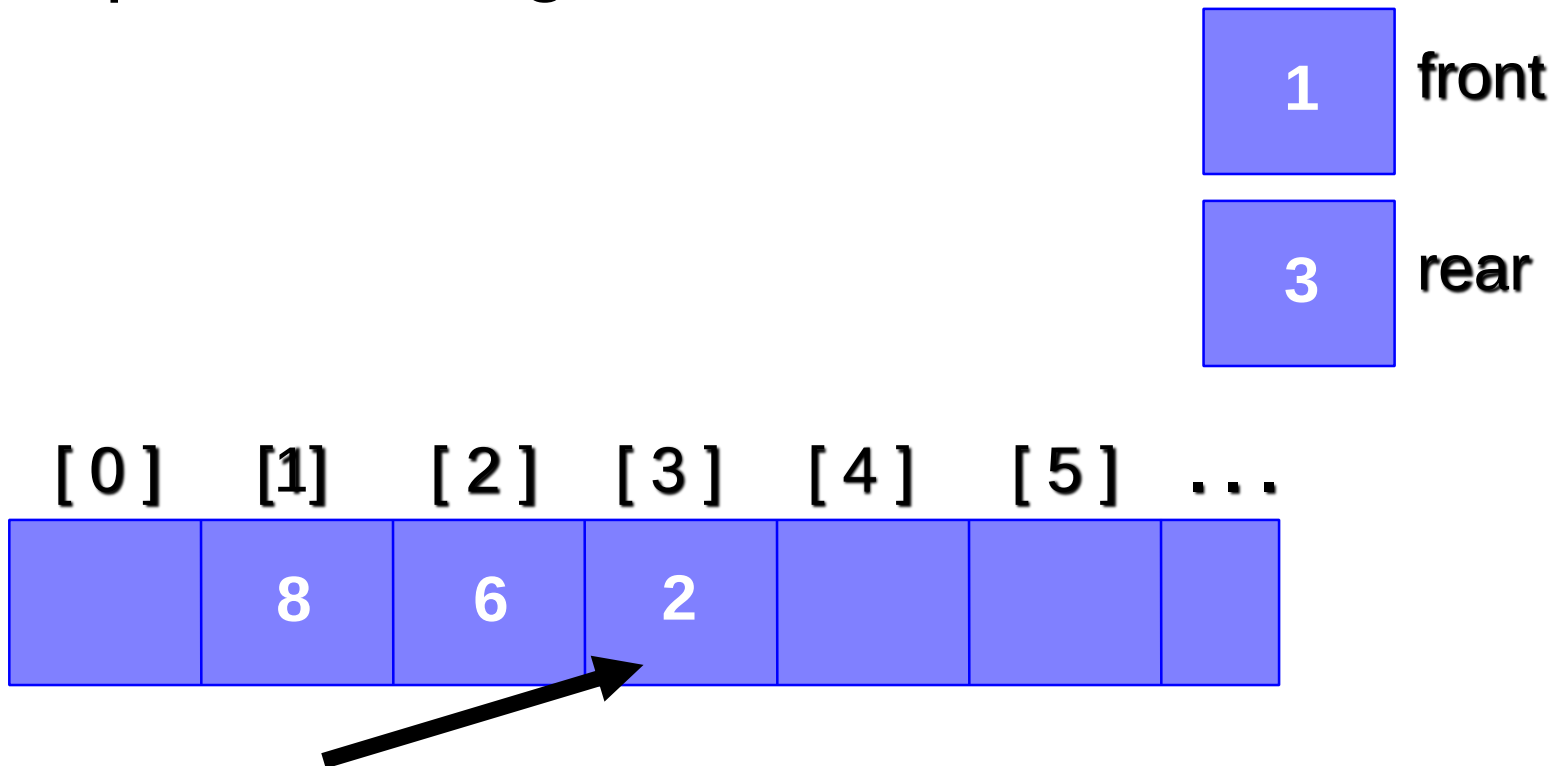
- When an element leaves the queue, change front





Enqueue Operation

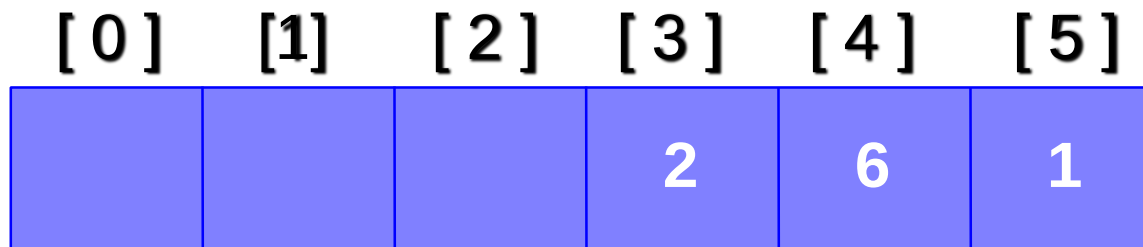
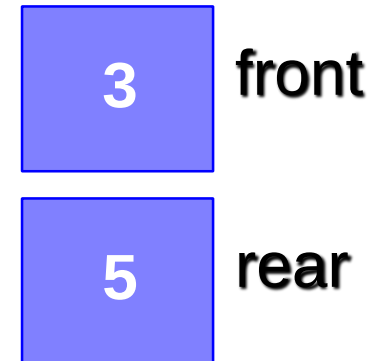
- When an element enters the queue change rear





At the end of Array!!

- Suppose we want to add a new element to this queue, size of array is 5.



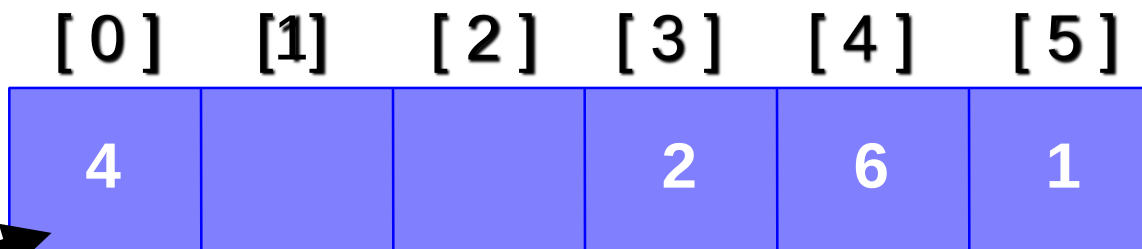


At the end of Array!! (2)

- The new element goes at the front of the array (if that slot is free):

3 front

0 rear





Array Implementation

- Easy to implement
- Limited capacity with a fixed array Or you must use a dynamic array for an unbounded capacity
- Special behaviour is needed when the rear reaches the end of the array.



Implementing Q using arrays

1. Declare array
2. Function to initialize queue
3. Function to check if queue is empty
4. Function to check if queue is full
5. Enqueue
6. Dequeue
7. Use the queue – enqueue and dequeue from main()



Implementing Q using arrays

1. Declare array
2. Function to initialize queue
void initialize(Queue queue)*
3. Function to check if queue is empty
int isEmpty(Queue queue)*
4. Function to check if queue is full
int isFull(Queue queue)*



Implementing Q using arrays

1. Declare array
2. Function to initialize queue
3. Function to check if queue is empty
4. Function to check if queue is full
5. Enqueue
void enqueue(Queue queue, int value)*



Implementing Q using arrays

1. Declare array
2. Function to initialize queue
3. Function to check if queue is empty
4. Function to check if queue is full
5. Enqueue
6. Dequeue
int dequeue(Queue queue)*



Implementing Q using arrays

1. Declare array
2. Function to initialize queue
3. Function to check if queue is empty
4. Function to check if queue is full
5. Enqueue
6. Dequeue
7. Use the queue – enqueue and dequeue from main()



Implementing Q using linked list

```
int main() {  
    Queue myQueue;  
    initialize(&myQueue);  
    enqueue(&myQueue, 10);  
    enqueue(&myQueue, 20);  
    enqueue(&myQueue, 30);  
    dequeue(&myQueue);  
    dequeue(&myQueue);  
    dequeue(&myQueue);  
    return 0;  
}
```



Implementing Q using linked list

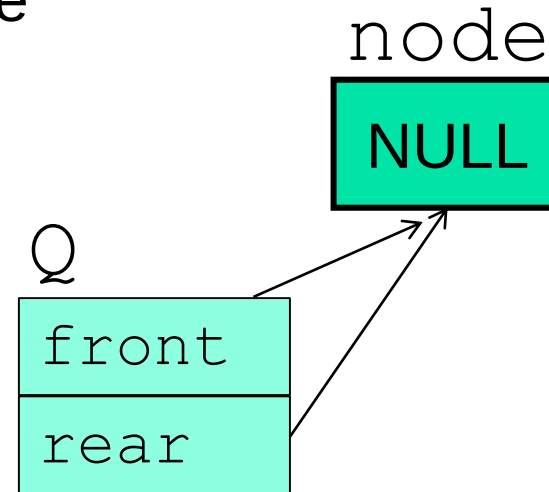
1. Declare queue structure

```
struct node {  
    int data;  
    struct node* next;  
};  
typedef struct node Node;  
typedef struct {  
    Node *front, *rear;  
}Queue;
```

Impl. of Q using Linked List

- Initialize an Empty Queue

```
void initialize(Queue *queue) {  
    queue->front = queue->rear = NULL;  
}
```



- Check if queue is empty

```
int isEmpty(Queue *queue){  
    return (queue->front == NULL && queue->rear == NULL);  
}
```

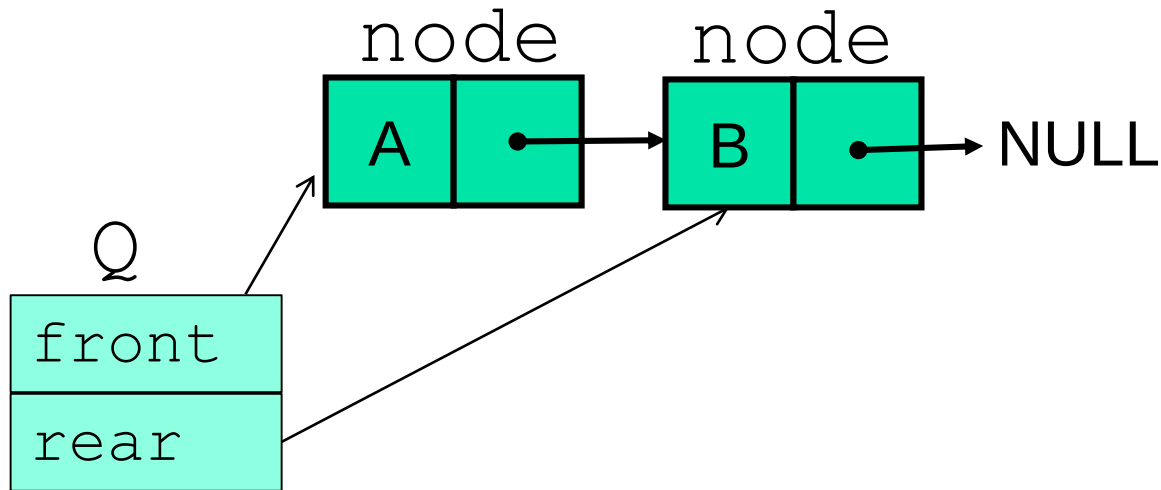


Implementing Q using linked list

1. Declare queue structure
2. Function to initialize queue
3. Function to check if queue is empty
4. **Enqueue**

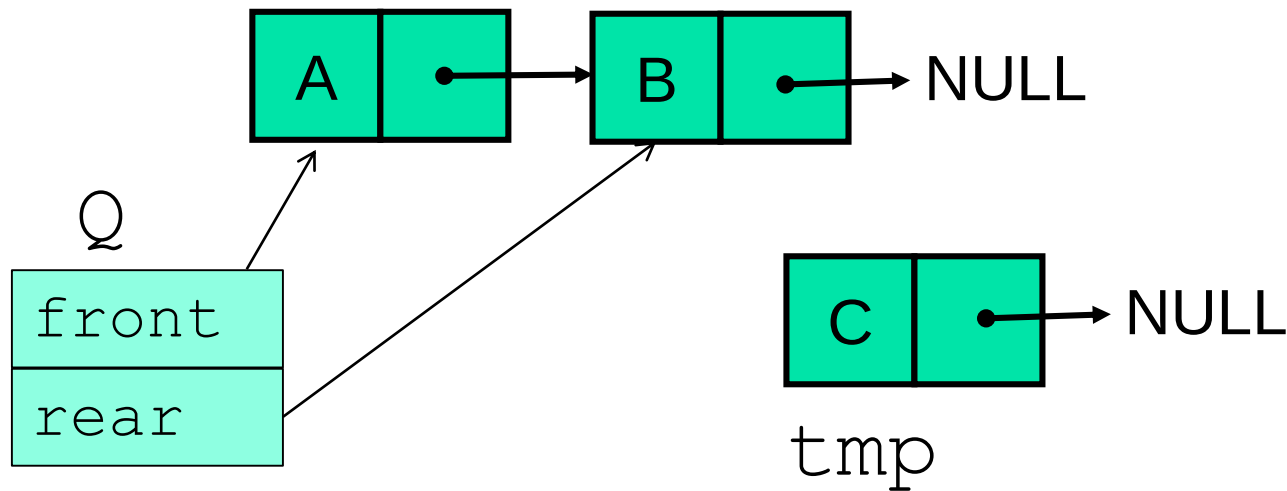
Enqueue (1)

- Queue with 2 nodes
 - Insert a node at the back of the queue



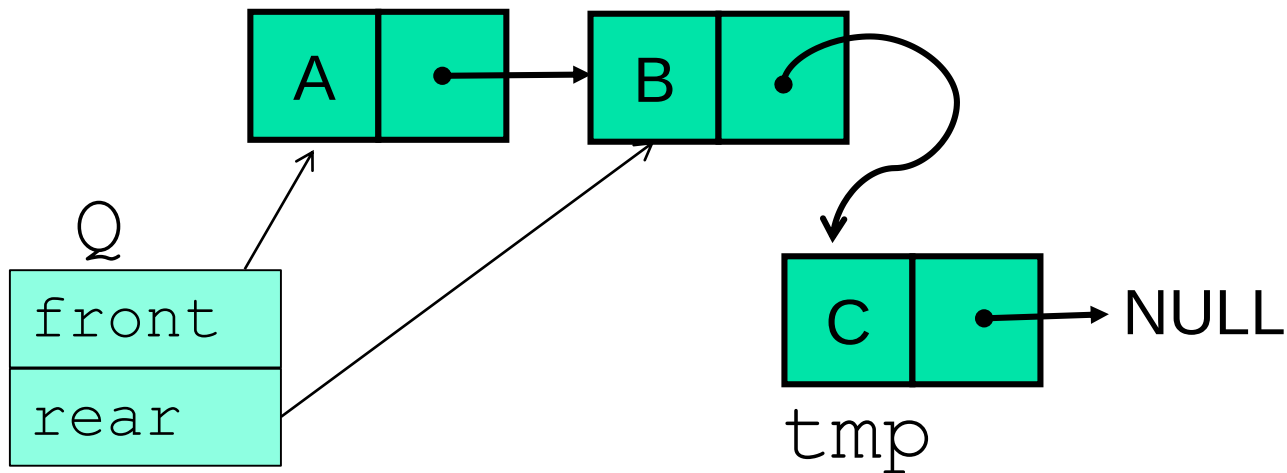
Enqueue

- Create a temporary node (tmp). Set tmp->data and tmp->next to NULL



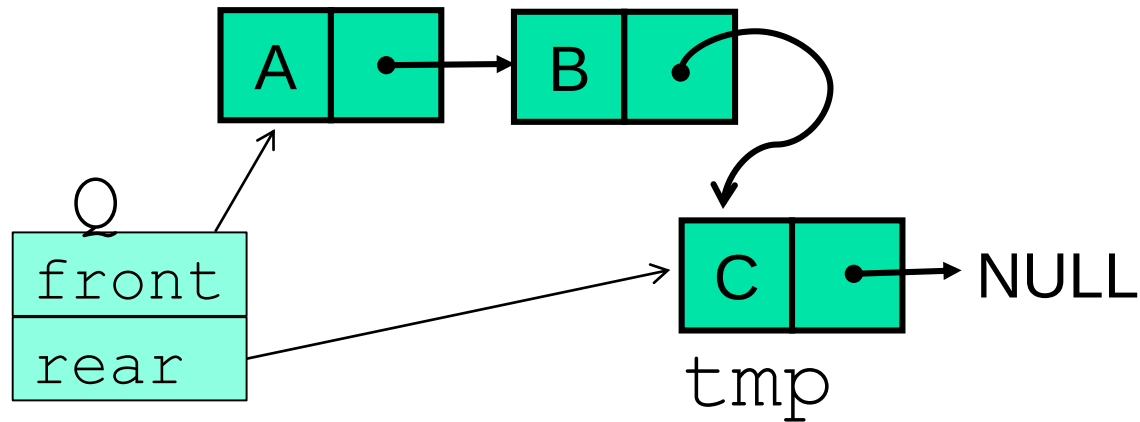
Enqueue

- Set $Q \rightarrow \text{rear} \rightarrow \text{next}$ to tmp



Enqueue

- Set $Q \rightarrow \text{rear}$ to tmp and increment count



- If Q is empty both front and rear will point to node



```
void enqueue(Queue* queue, int item) {
    Node* node = (Node *) malloc(sizeof(Node));
    if (node == NULL) {
        perror("Queue overflow");
        exit(EXIT_FAILURE);
    }
    node->data = item;
    node->next = NULL;
    if (isEmpty(queue))
        queue->front = queue->rear = node;
    else {
        queue->rear->next = node;
        queue->rear = node;
    }
}
```

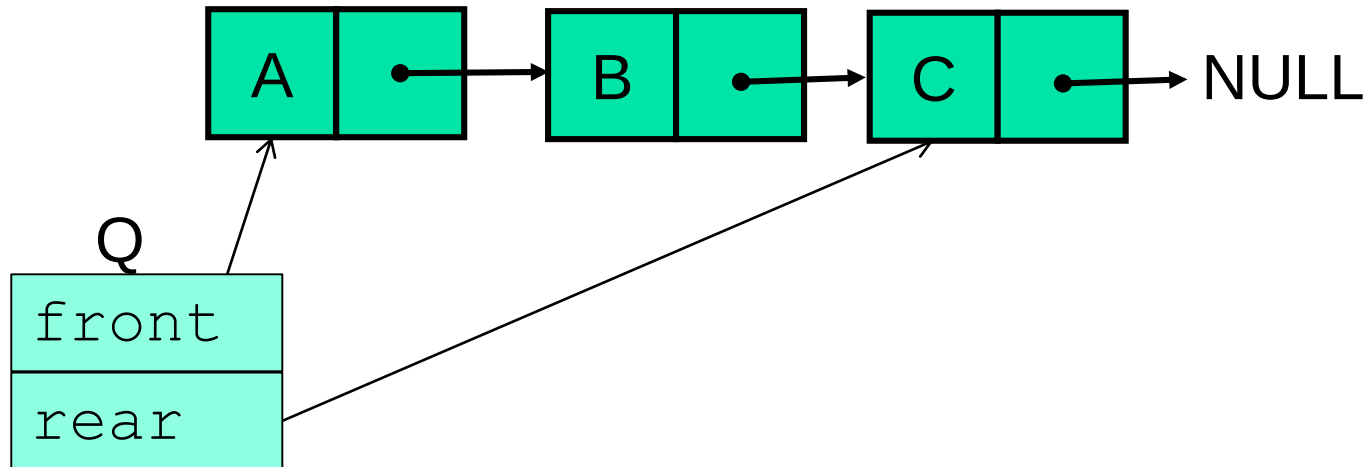


Implementing Q using linked list

1. Declare queue structure
2. Function to initialize queue
3. Function to check if queue is empty
4. Enqueue
5. Dequeue
6. Use the queue – enqueue and dequeue from main()

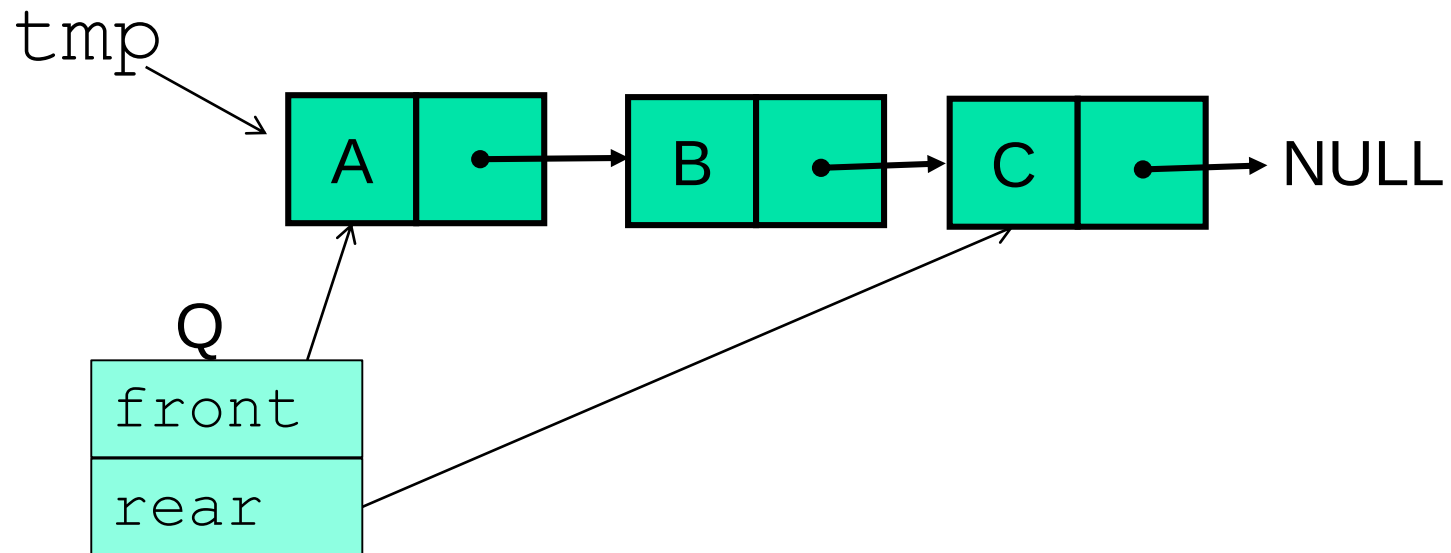
Dequeue

- Delete a node at the front of the queue



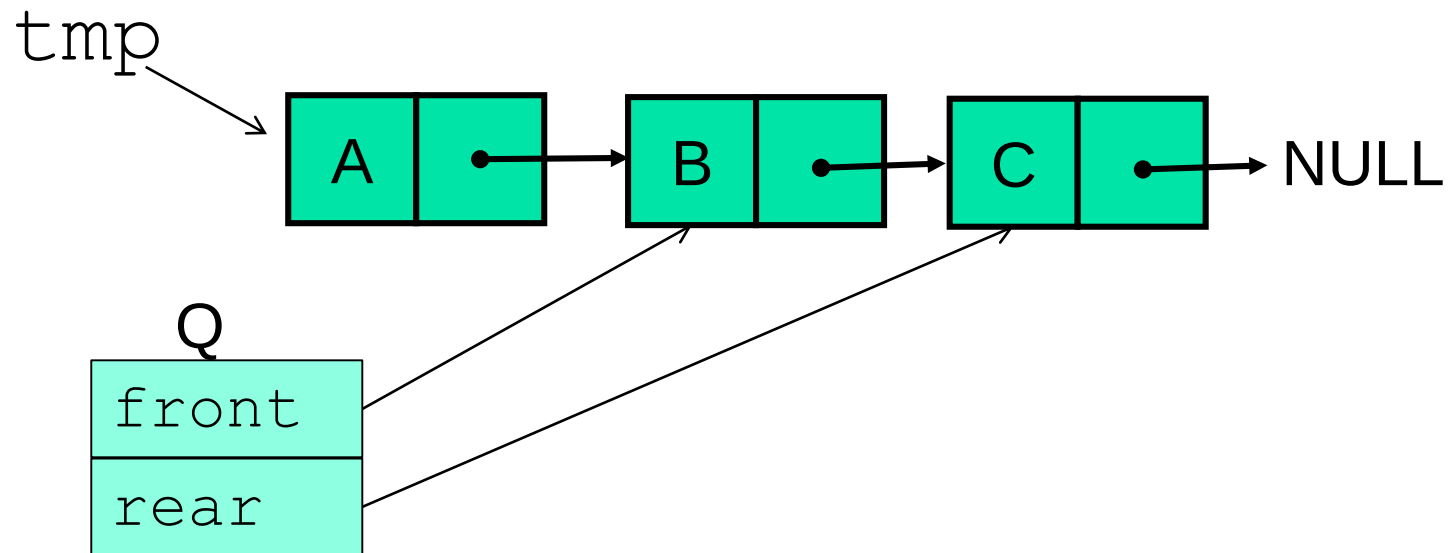
Dequeue

- Set tmp to $Q \rightarrow \text{front}$



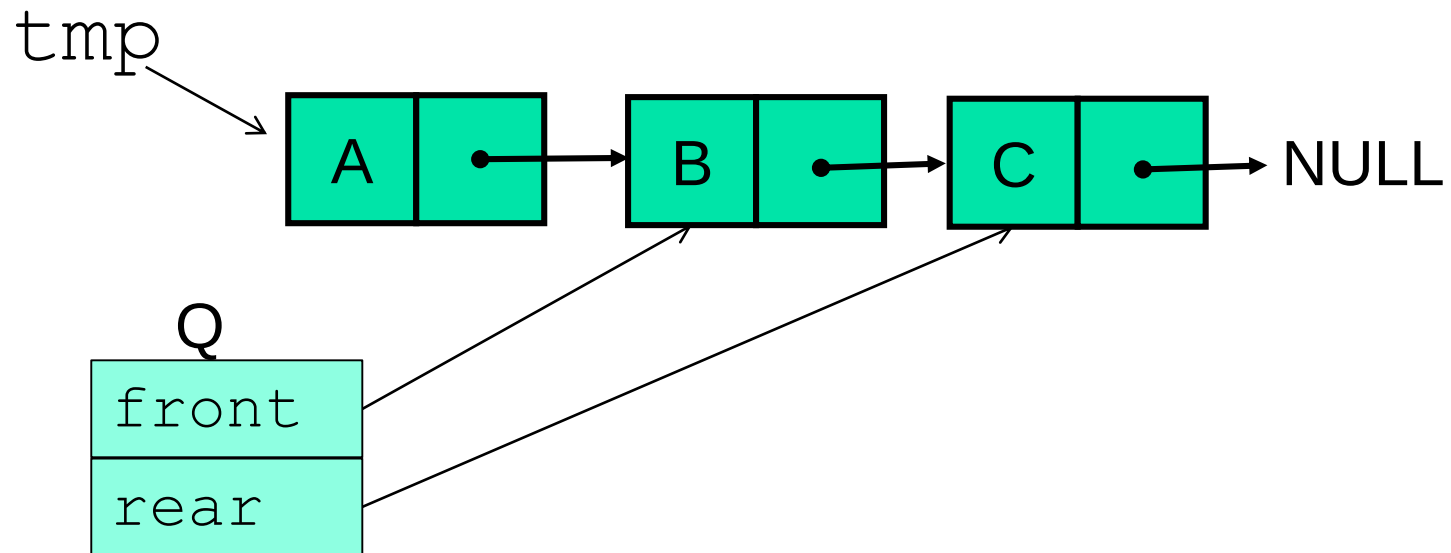
Dequeue

- Set $Q \rightarrow \text{front}$ to $Q \rightarrow \text{front} \rightarrow \text{next}$



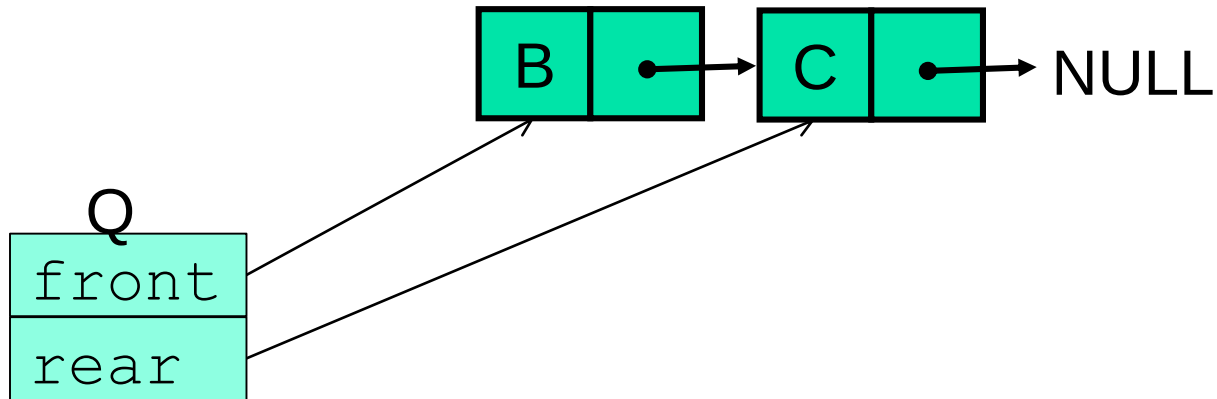
Dequeue

- `free(tmp)`



Dequeue

- `free(tmp)`



- If `q->front` becomes `NULL`, means `Q` is empty, then `rear` will also point to `NULL`



```
int dequeue(Queue* queue) {
    if (isEmpty(queue))
        return -1;
    Node *tmp = queue->front;
    int item = tmp->data;
    queue->front = tmp->next;
    free(tmp);
    if (queue->front == NULL) // If the Q becomes empty
        queue->rear = NULL;
    return item;
}
```



Implementing Q using linked list

1. Declare queue structure
2. Function to initialize queue
3. Function to check if queue is empty
4. Enqueue
5. Dequeue
6. Use the queue – enqueue and dequeue from main()